

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 3

- Due: **11:59 PM** on **Monday, April 6**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Make sure to **rename your notebook file** as instructed.

Slide update

- The second slide on page 13 of last class's slides has been revised.
- The notebook file (ics604-18-19_correlation.ipynb) uploaded today reflects this update.
 - The previous file (ics604-18_correlation.ipynb) has been removed.

Lecture 19

- Correlations
 - Coefficient of determination (R^2)
 - Spearman's rank correlation (ρ)
- Linear Regression

R^2 and the explained variance

- R^2 (**coefficient of determination**) is the square of the Pearson's correlation coefficient R .
- The interpretation of R^2 is different from that of R .
 - Measures the proportion of variance explained by the linear relationship.
 - Takes values between 0 and 1, regardless of the direction of the relationship.
- Example:
 - If $R = 0.8$, then $R^2 = 0.64$.
⇒ 64% of the variability is explained, 36% remains unexplained.

Example

- The number of hotel rooms sold in Waikiki and average daily sales at ABC stores also in Waikiki.

```
daily_sales = pd.read_table("data/room_abc_sales.tsv", header=None)
daily_sales
```

	0	1
0	180	66649.07
1	181	71557.20
2	182	104553.23
3	183	146851.52
4	184	84260.95
...	...	...
2815	2995	153410.75
2816	2996	170248.67
2817	2997	158253.04
2818	2998	182130.50
2819	2999	157206.72

2820 rows × 2 columns

Question

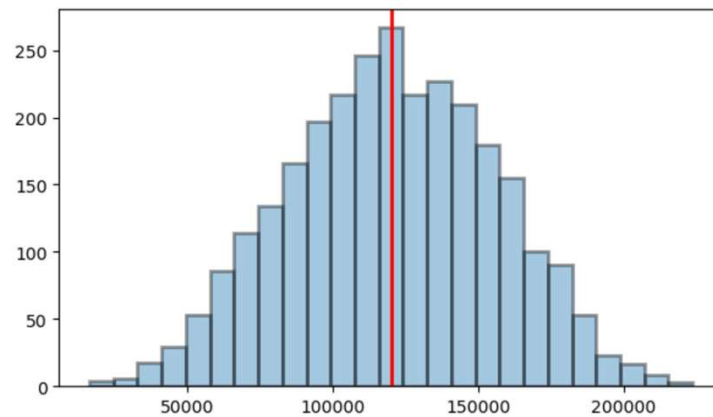
- In the absence of any other information about ABC store sales, what is your prediction for next Tuesday's sales?

```
print(daily_sales[1].mean())
```

```
120227.70596808512
```

```
plt.figure(figsize=(7, 4))
```

```
plt.hist(daily_sales[1], bins=25, edgecolor="k", linewidth=2, alpha=0.4)  
plt.axvline(daily_sales[1].mean(), color='r', linewidth=2);
```

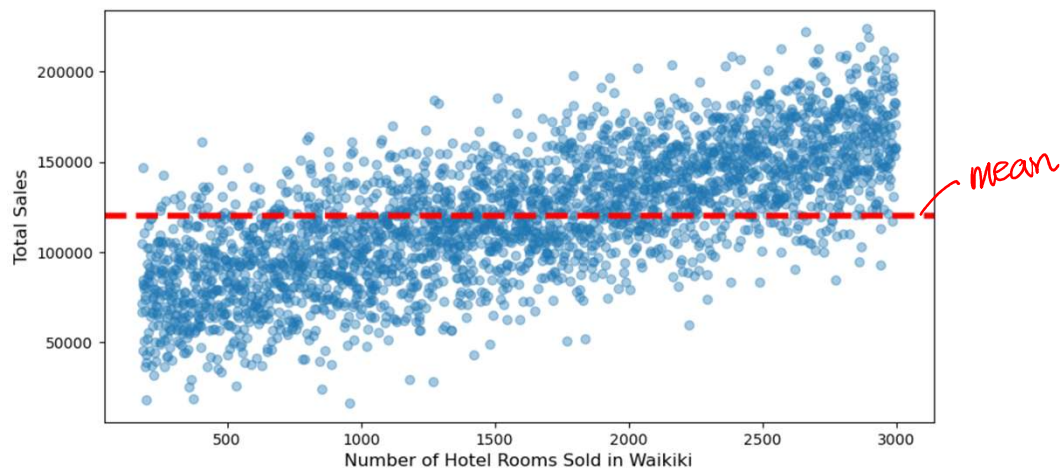


Sales distribution

- Daily sales are assumed to follow a normal distribution with mean $\mu = \$120,228$.
- In the absence of additional information, the mean provides a reasonable baseline prediction for the daily sales.
 - “Reasonable” here refers to a choice among constant (uninformed) predictions, though the optimal choice depends on the loss function used.
- What if additional relevant information is available?

```
plt.figure(figsize=(10, 5))

plt.scatter(daily_sales[0], daily_sales[1], alpha=0.4)
plt.xlabel("Number of Hotel Rooms Sold in Waikiki", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.axhline(np.mean(daily_sales[1]), color='r', linestyle='--', linewidth=4);
```

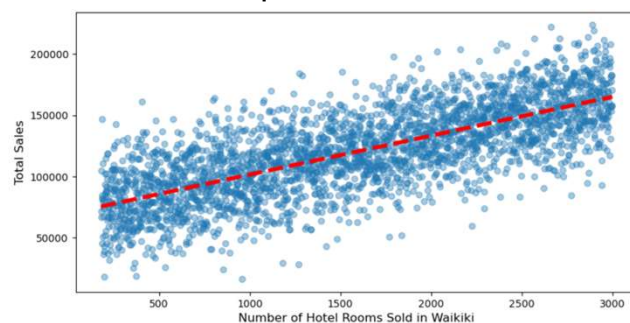


Sales distribution (cont'd.)

- Prediction accuracy can improve when incorporating additional relevant information, such as:
 - Number of hotel rooms sold near ABC stores
 - Presence of special events
 - Holidays or seasonal factors
- I.e., variables correlated with sales can lead to more accurate, data-informed predictions than a constant mean.

Regression line

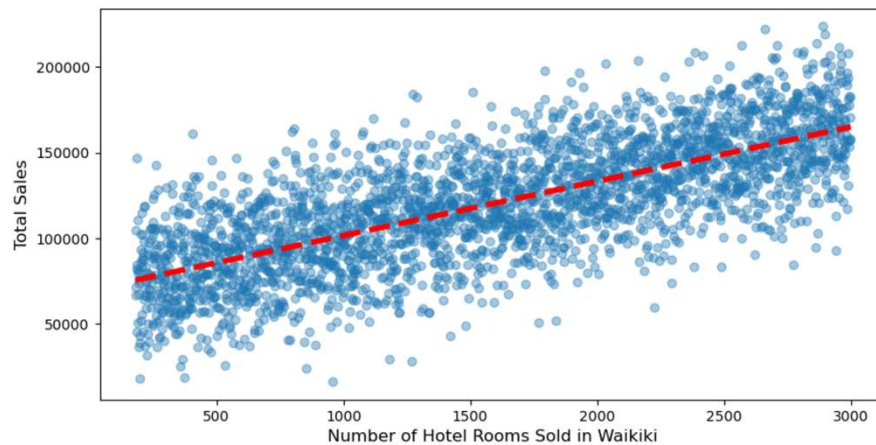
- In the presence of hotel rooms sold, the daily sales value can be represented by the regression line.
 - The line which best fits the data.
- The regression line is the model we believe explains the data.
 - How sales change as the hotel occupancy changes



```
plt.figure(figsize=(10, 5))

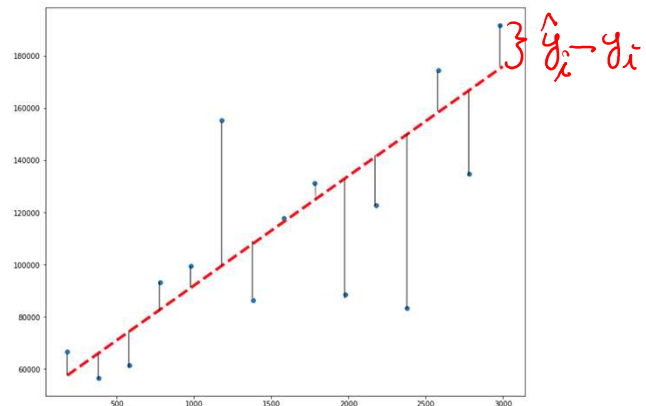
plt.scatter(daily_sales[0], daily_sales[1], alpha=0.4)
plt.plot(daily_sales[0], 69843.88 + 31.69 * daily_sales[0], color='r', linewidth=4, linestyle='--')

plt.xlabel("Number of Hotel Rooms Sold in Waikiki", fontsize=12)
plt.ylabel("Total Sales", fontsize=12);
```



What are the residuals?

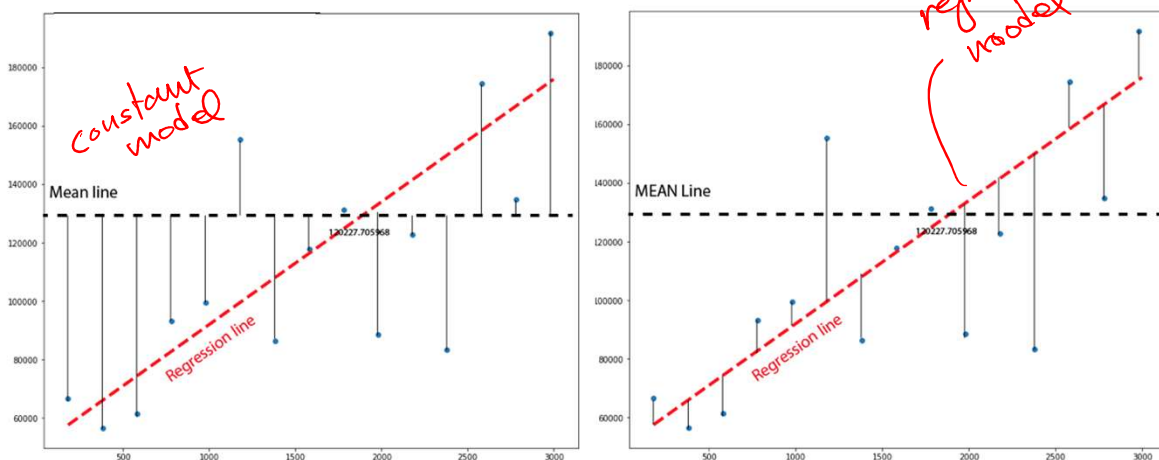
- A residual represents the difference between an observed value and a value estimated using the regression line.
 - The error between the observed value and the predicted value.



Minimizing the residuals

- Naturally, when predicting a quantity of interest, we want to find the method that minimizes the residuals.
- The total sum of all the residuals should be as small as possible.
- Therefore, the model (or the red line in the plot) should match the data points as closely as possible, ensuring that the residuals are minimal.
- What if we used the mean for our predictions?

Mean vs. Regression residuals



Mean vs. Regression residuals (cont'd.)

- The variance around the mean is just the natural variance in your data.
- The regression line is the line that best fits the data, i.e., minimizes the total sum of residuals.
- Visually, we can confirm that the regression line fits the data better than the mean (black line in the previous slide).

Mean vs. Regression residuals (cont'd.)

- R^2 quantifies the difference between the best fitting line (the one that minimizes the sum of residuals) and the mean line.

$$\frac{SSR_{mean} - SSR_{regression}}{SSR_{mean}}$$

Sum of Squared Residuals (SSR)

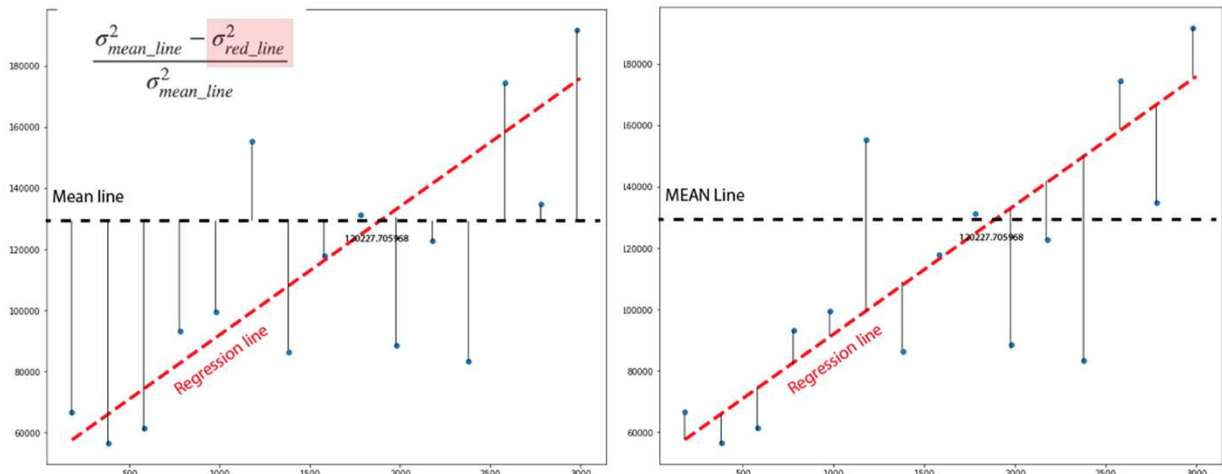
- We normalize by SSR_{mean} so that the R^2 is constrained to the interval $[0, 1]$.

- $SSR_{regression}$ will never be greater than SSR_{mean} .

- Difference between the variance around the mean line and the variance around the regression line, normalized by the variance around the mean line.

$$\rightarrow \frac{0.8 - 0.5}{0.8} = \frac{0.3}{0.8} \approx 0.37$$

Mean vs. Regression residuals (cont'd.)



```
from scipy import stats
```

```
print(stats.pearsonr(daily_sales[0], daily_sales[1]))
```

```
print(stats.pearsonr(daily_sales[0], daily_sales[1])[0] ** 2)
```

```
PearsonRResult(statistic=np.float64(0.7262702431699775), pvalue=np.float64(0.0))
0.5274684661141782
```

```
var_mean = np.sum((daily_sales[1] - np.mean(daily_sales[1]))**2)
print(var_mean)
```

```
3559841164665.26
```

```
predicted_val = 69843.88 + 31.69 * np.array(daily_sales[0])
```

```
var_red_line = np.sum((daily_sales[1] - predicted_val)**2)
```

```
print(var_red_line)
```

```
1682137768328.9448
```

```
print((var_mean - var_red_line) / var_mean)
```

```
0.5274683081296634
```

rooms sold

$= R^2$

R

SSR_{MEAN}

SSR_{reg.}

manually calculate R^2

Variance explained

- The data show a 0.726 correlation between the number of hotel rooms sold and the ABC store's daily sales.
 - $R = 0.726$ then $R^2 = R \times R = 0.527$.
- Regression line explains approximately 53% of the variance in the data.
 - We understand 53% of what drives the sales at the ABC store.
- In the absence of additional explanatory variables, the number of hotel rooms sold in Waikiki is a good predictor of total sales in Waikiki's ABC store.
 - Additional explanatory variables could perhaps explain more variance in the ABC store's daily sales.
- It certainly is better than just taking the mean as a predictor.

Significance and the variance explained

- If the variance explained (R^2) is low, then X is likely not a strong predictor of Y in the context of a linear model.
 - This does not rule out the possibility of a non-linear relationship.
- In practice, multiply explanatory variables are often available and can improve predictions.
 - Incorporating additional relevant variables can increase explained variance and reduce residual error.
- For example, when modeling ABC store sales:
 - Weather conditions (e.g., rain or cold) may reduce tourist shopping activity.
 - Local events in Waikiki may increase sales.
 - Combining these factors, along with hotel occupancy, can lead to a more accurate and informative model.

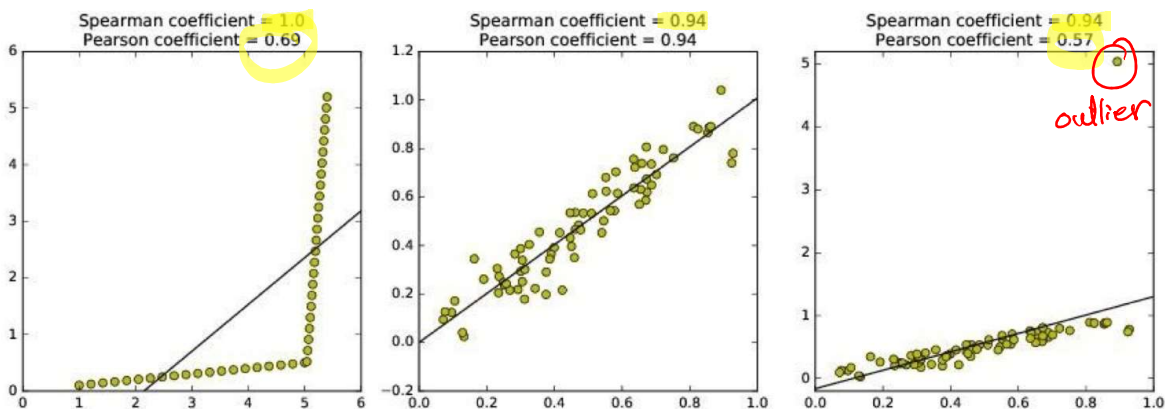
Spearman's rank correlation

↪ monotonic relationship may not be linear

- The Spearman's rank correlation is similar to the Pearson's correlation coefficient but with rank data.
- Counts the number of disordered pairs, not how well the data fits a line.
- Better with skewed distribution and outliers.
- Let $rank_{x_i}$ be the rank position of x_i in sorted order.
 - $rank_{x_i}$ ranges from 1 to n .

$$\rho = 1 - \frac{6 \sum (rank_{x_i} - rank_{y_i})^2}{n(n^2 - 1)}$$

Spearman's rank correlation (cont'd.)



Note that Pearson's correlation coefficient measures the strength of a linear association, not whether or not the data have a linear association.

Correlation and causation

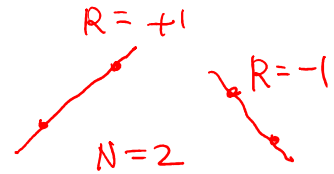
- Correlation does not imply causation.
- Examples:
 - The number of policemen active in a precinct correlated strongly with the local crime rate, but the police do not cause the crime.
 - If we collect data for the total number of Master's degrees issued by universities each year and the total box office revenue generated by year, we would find that the two variables are highly correlated.
 - Global population has been increasing each year.
 - Leading to more Master's degrees being issued.
 - Leading to more people attending movies each year.
 - Both increase in roughly equal amounts.

<https://www.statology.org/correlation-does-not-imply-causation-examples/>

Statistical significance

- Data miners are primarily concerned with whether their observations are interesting.
- Statisticians are primarily concerned with whether observations on data are significant.
 - As we discussed, something is statistically significant if the odds of it happening by chance are rare.
 - Less than 1 in 20 (probability of the event is less than 0.05).

Interpreting correlation significance



- Statistical significance of a correlation measures how likely it is for data to be correlated just by chance.
- Depends on the sample size and the real underlying correlation coefficient.
 - Two data points are always correlated ($R = 1$ or -1).
 - Much larger dataset are less likely to be correlated just by chance.
- Using bootstrapping, we can build our own background distribution of correlation coefficients, which we can use to assess the statistical significance of any given correlation value.

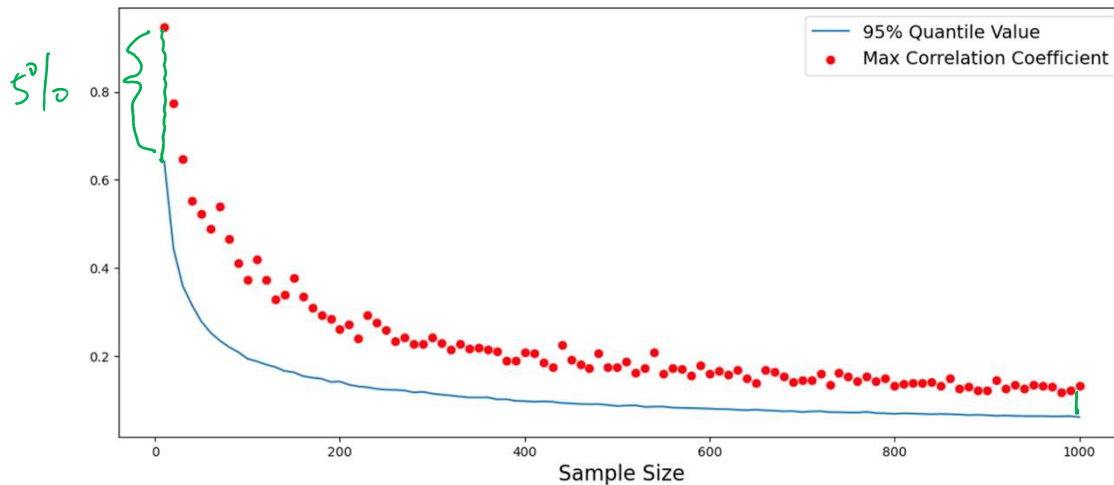
```
def simulate_corr(n=10, nb_repeats=1000, p=1/20):  
    corr_ = []  
  
    for _ in range(nb_repeats):  
        x = np.random.randint(0, 100, n)  
        y = np.random.randint(0, 100, n)  
        corr_.append(np.abs(stats.pearsonr(x, y)[0]))  
    return (np.quantile(corr_, 1-p), max(corr_))
```

```
simulate_corr(n=10, nb_repeats=100000, p=1/20)
```

```
(0.6339153082702135, 0.9798516159933374)
```

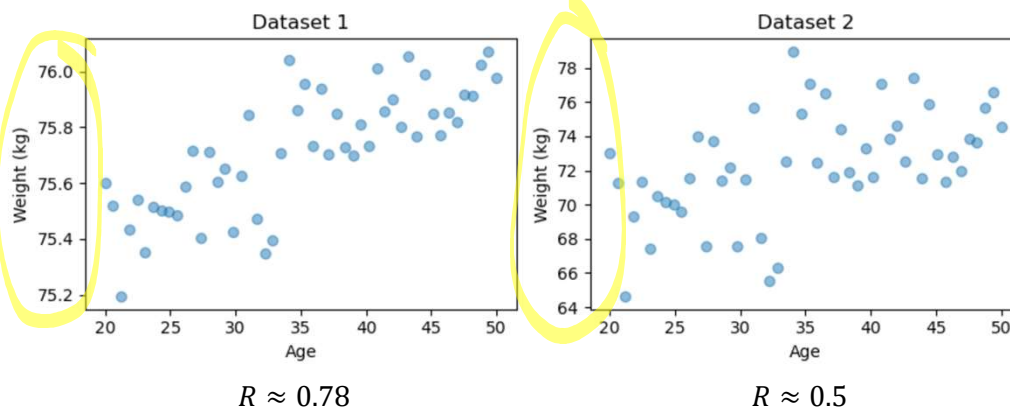
```
sample_size = np.arange(10, 1010, 10)  
max_common_corr = []  
max_corr = []  
  
for n in sample_size:  
    max_c_r, max_r = simulate_corr(n, nb_repeats=10000)  
    max_common_corr.append(max_c_r)  
    max_corr.append(max_r)
```

```
plt.figure(figsize=(14, 6))
plt.plot(sample_size, max_common_corr, label=f"95% Quantile Value")
plt.scatter(sample_size, max_corr, color="red", label="Max Correlation Coefficient")
plt.xlabel("Sample Size", fontsize=16)
plt.legend(fontsize=14);
```



Correlation vs. Practical importance

- A higher correlation does not necessarily imply a more meaningful or impactful relationship.



Linear Regression

Simple linear regression

- Also known as single linear regression.
- The regression equation models the relationship between the magnitude of one variable and that of a second as a line.
 - As we discussed correlation is another way to measure how two variables are related.
- The difference is that while correlation measures the strength of an association between two variables, regression quantifies the nature of the relationship.

Simple linear regression (cont'd.)

- A simple method to predict a *quantitative* response Y on the basis of a single predictor variable X .
- Assume an approximately linear relationship between X and Y .
- Remember that the equation of a line is:

$$Y = \beta_0 + \beta_1 X$$

- Regression models are typically fit by the method of least squares.

Simple linear regression: Key terms $Y = \beta_0 + \beta_1 X$

- **Response variable** (or dependent variable, target, outcome): The variable we are trying to predict. Y
- **Independent variable** (or feature, attribute, predictor): The variable used to predict the response. X
- **Intercept** (often denoted b_0 or β_0): The intercept of the regression line, i.e., the predicted value when $X = 0$.
- **Regression coefficient** (often denoted b_1 or β_1): The slope of the regression line.
 - β_0 and β_1 are often called parameter estimates, or weights.
- **Fitted values** (or predicted values): The estimates $\hat{Y}_i$ obtained from the regression line.
- **Residuals** (or errors): The difference between the observed values and the fitted values.
- **Least squares** (ordinary least squares, OLS) : The method of fitting a regression by minimizing the sum of squared residuals. $Y_i - \hat{Y}_i$

Example: Advertising data

- Dataset shows **product sales** (in thousands of units) and **advertising budgets** (in thousands of dollars) across TV, radio, and newspaper.
- Which advertising medium contributes most to sales?
- How accurately can we estimate the effect of each medium?
- Do combinations of media produce synergistic effects beyond their individual contributions?

```
advertising = pd.read_csv('data/Advertising.csv', usecols=[1, 2, 3, 4])  
advertising.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

Linear regression

```
from scipy.stats import linregress
```

```
linreg = linregress(advertising["TV"], advertising["sales"])  
linreg
```

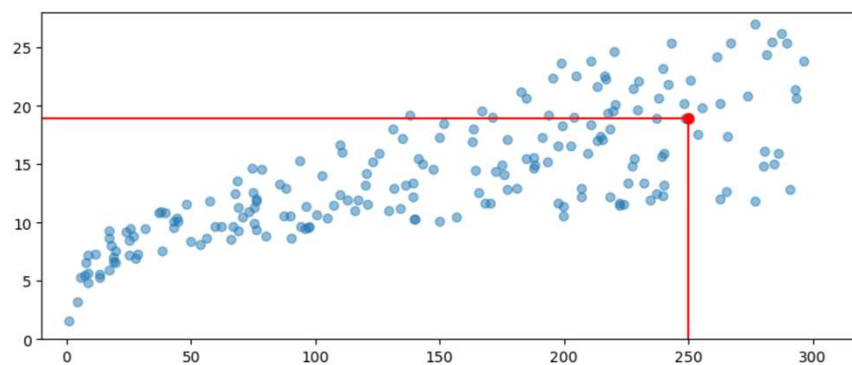
LinregressResult(slope= β_1 , intercept= β_0 , rvalue= R , pvalue=1.4673897001948012e-42, stderr=0.0026906071877968703, intercept_stderr=0.4578429402734785)

- `linregress` returns the intercept β_0 and the slope β_1 of the linear relationship between sales and advertising budget for TV:

$$\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}$$

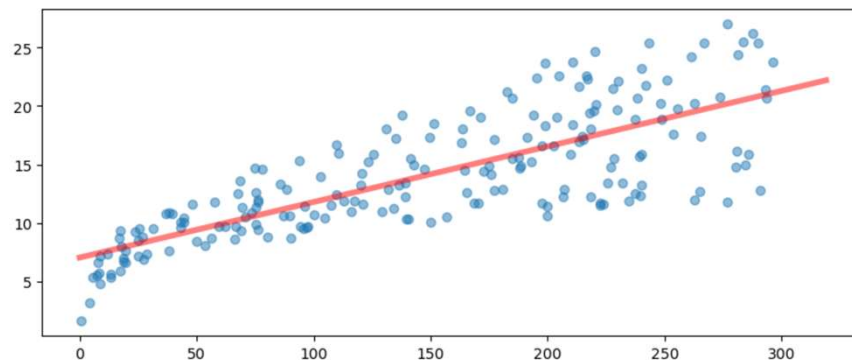
- Together, β_0 and β_1 are known as the model parameters.

```
some_tv_advertising = 250  
some_sales_prediction = linreg.intercept + linreg.slope * some_tv_advertising  
  
plt.figure(figsize=(10, 4))  
plt.scatter(advertising["TV"], advertising["sales"], alpha=0.5)  
plt.xlim(-10, 320)  
plt.ylim(0, 28)  
plt.vlines(some_tv_advertising, 0, some_sales_prediction, color='red')  
plt.hlines(some_sales_prediction, -10, some_tv_advertising, color='red')  
plt.scatter(some_tv_advertising, some_sales_prediction, s=50, color='red');
```



```
x_axis = np.arange(0, 320)
y_line = linreg.intercept + linreg.slope * x_axis

plt.figure(figsize=(10, 4))
plt.scatter(advertising["TV"], advertising["sales"], alpha=0.5)
plt.plot(x_axis, y_line, color='red', linewidth=4, alpha=0.5);
```



The quality of the regression

```
>>> linregress(advertising["TV"], advertising["sales"])
```

```
LinregressResult(slope=0.0475366404330197, intercept=7.032593549127702,
rvalue=0.7822244248616063, pvalue=1.4673897001948012e-42,
stderr=0.0026906071877968703, intercept_stderr=0.4578429402734785)
```

- The correlation coefficient for the relationship between TV and sales is rather high.
 - TV advertising budget explains $R^2 \approx 0.61$ of the variance in the product's sales.
- Recall that the error is the difference between the predicted value and the actual value.
 - If $\hat{y}_i = \beta_0 + \beta_1 x_i$, then the error for i^{th} data point is $e_i = \hat{y}_i - y_i$.

```
some_point = advertising.iloc[0]
some_point
```

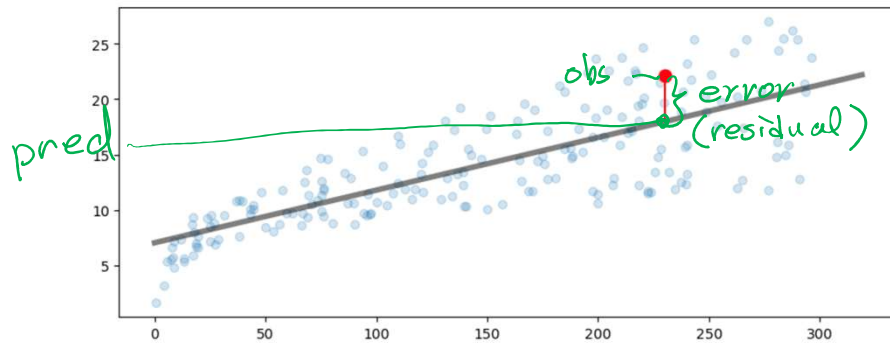
```
TV          230.1
radio        37.8
newspaper    69.2
sales        22.1
Name: 0, dtype: float64
```

```
x_axis = np.arange(0, 320)
y_line = linreg.intercept + linreg.slope * x_axis
```

```
plt.figure(figsize=(10, 4))
plt.scatter(advertising["TV"], advertising["sales"], alpha=0.2)
plt.plot(x_axis, y_line, color='black', linewidth=4, alpha=0.5)
```

```
y_hat = linreg.intercept + linreg.slope * some_point["TV"]
```

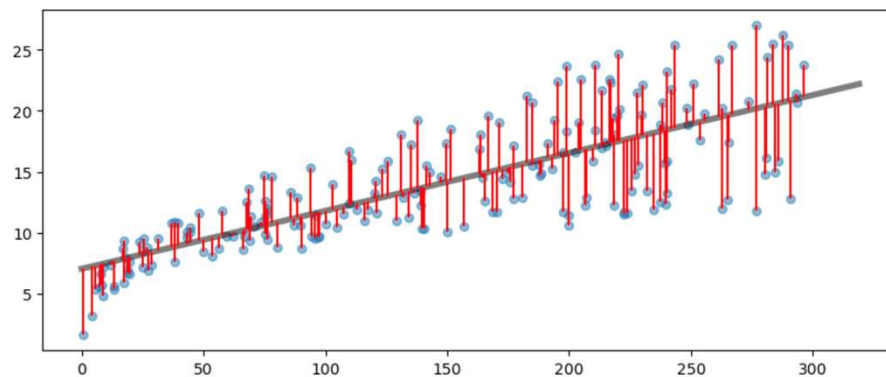
```
plt.scatter(some_point["TV"], some_point["sales"], color='red', s=80)
plt.vlines(some_point["TV"], some_point["sales"], y_hat, color='red');
```



```
x_axis = np.arange(0, 320)
y_line = linreg.intercept + linreg.slope * x_axis
```

```
plt.figure(figsize=(10, 4))
plt.scatter(advertising["TV"], advertising["sales"], s=30, alpha=0.5)
plt.plot(x_axis, y_line, color='black', linewidth=4, alpha=0.5)
```

```
for i in range(advertising.shape[0]):
    y_hat = linreg.intercept + linreg.slope * advertising.iloc[i]["TV"]
    plt.vlines(advertising.iloc[i]["TV"], advertising.iloc[i]["sales"], y_hat, color='red')
```

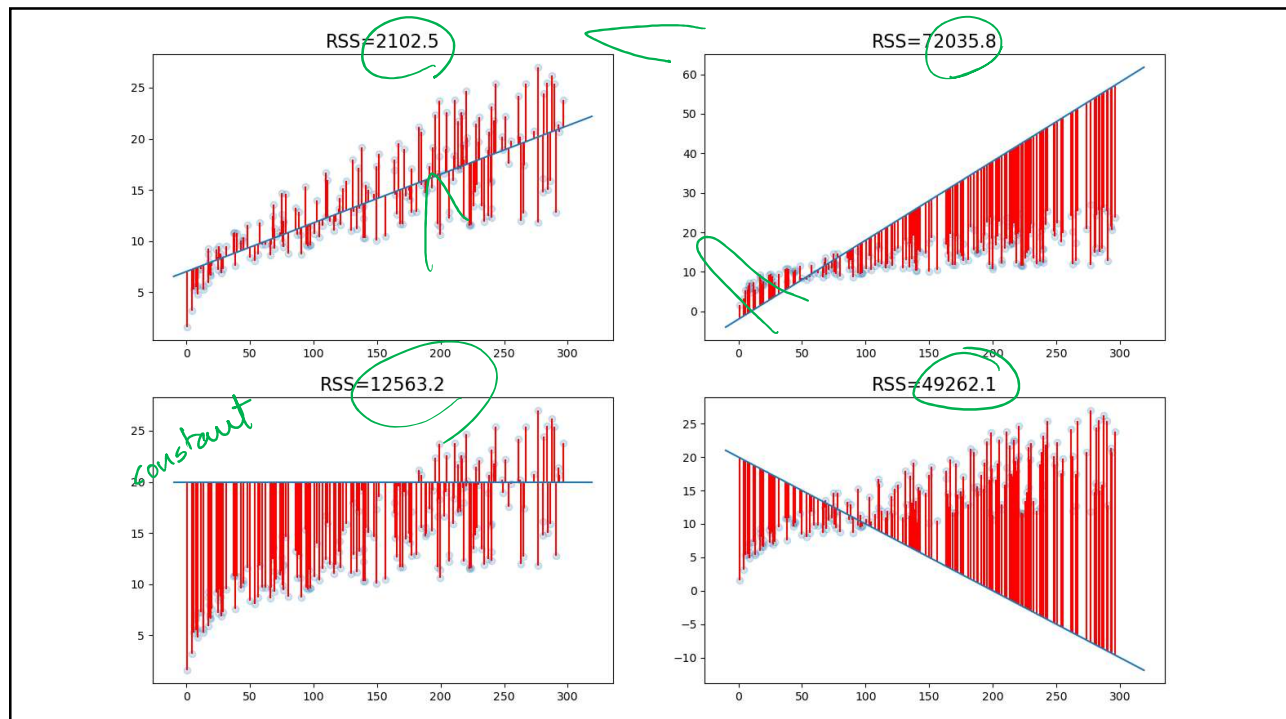


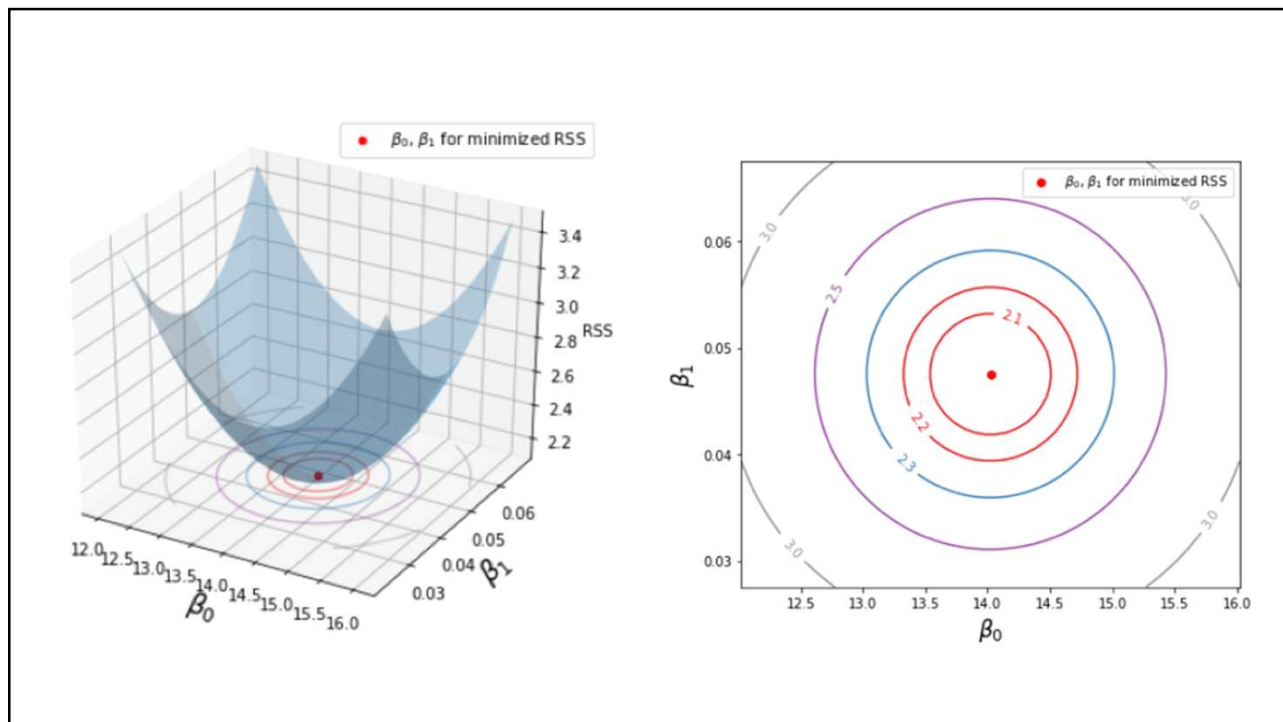
The residual sum of squares

- We can quantify how well a line fits the data simply by assessing the total error.
- To avoid the error values cancel out, we compute the sum of square of the error.
- We call that the residual sum of squares (RSS).

$$RSS = e_1^2 + e_2^2 + \dots + e_n^2$$

- Naturally, we want the linear regression model that provides the smallest RSS.
- The line with the smallest RSS is the best model under the current assumption of a linear relationship.





Minimizing the RSS

- For linear regression, we can use calculus to arrive at the closed form solutions of β_0 and β_1 and that minimize the RSS.
- Minimize equation by finding where partial derivatives are equal to 0:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad \frac{\partial \text{RSS}}{\partial \beta_1} = 0$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} \quad \frac{\partial \text{RSS}}{\partial \beta_0} = 0$$

- For more complex models, we may resort to iterative optimization algorithms to find the best parameters.
 - E.g.: gradient descent, genetic algorithms, etc.

Questions

- Our data are samples from a larger population and getting new data may lead to different parameter estimates.
- How confident are we in our model estimates?
- How can we quantify our belief in our parameter estimates?

Confidence interval of model parameters

- One way we have done this in the past is using bootstrap.
 - Subsample with replacement the same number of points.
 - Compute the linear regression using the bootstrap sample.
 - Compute the 95% confidence interval of the model.


```
print(advertising.shape)
```

```
(200, 4)
```

```
# bootstrapping the data (same sample size)
```

```
advertising.sample(advertising.shape[0], replace=True).head()
```

	TV	radio	newspaper	sales
--	----	-------	-----------	-------

117	76.4	0.8	14.8	9.4
-----	------	-----	------	-----

60	53.5	2.0	21.4	8.1
----	------	-----	------	-----

158	11.7	36.9	45.2	7.3
-----	------	------	------	-----

193	166.8	42.0	3.6	19.6
-----	-------	------	-----	------

36	266.9	43.8	5.0	25.4
----	-------	------	-----	------

```
advertising.sample(advertising.shape[0], replace=True).head()
```

	TV	radio	newspaper	sales
--	----	-------	-----------	-------

78	5.4	29.9	9.4	5.3
----	-----	------	-----	-----

10	66.1	5.8	24.2	8.6
----	------	-----	------	-----

157	149.8	1.3	24.3	10.1
-----	-------	-----	------	------

141	193.7	35.4	75.6	19.2
-----	-------	------	------	------

58	210.8	49.6	37.7	23.8
----	-------	------	------	------

```
bootstrap_intercepts = []
```

```
bootstrap_slopes = []
```

```
for i in range(50_000):
```

```
    sample_indexes = np.random.choice(advertising.index, size=len(advertising.index))
```

```
    bootstrap_sample = advertising.iloc[sample_indexes]
```

```
    lr = linregress(bootstrap_sample["TV"], bootstrap_sample["sales"])
```

```
    bootstrap_intercepts.append(round(lr.intercept, 3))
```

```
    bootstrap_slopes.append(round(lr.slope, 3))
```

```
print("10 first intercepts:")
```

```
print(bootstrap_intercepts[:10])
```

```
print("10 first slopes:")
```

```
print(bootstrap_slopes[:10])
```

```
10 first intercepts:
```

```
[6.759, 7.254, 6.873, 7.102, 7.317, 7.269, 6.835, 7.22, 6.774, 6.971]
```

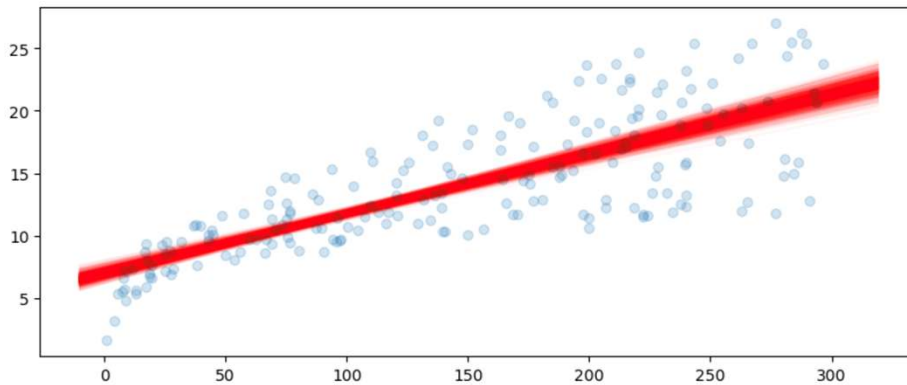
```
10 first slopes:
```

```
[0.05, 0.047, 0.049, 0.047, 0.044, 0.046, 0.048, 0.044, 0.051, 0.048]
```

```
plt.figure(figsize=(10, 4))
plt.scatter(advertising["TV"], advertising["sales"], alpha=0.2)

x_axis = np.arange(-10, 320)

for i in np.random.choice(range(50000), 1000, replace=False):
    y_hat = bootstrap_intercepts[i] + bootstrap_slopes[i] * x_axis
    plt.plot(x_axis, y_hat, color='red', alpha=0.02)
```



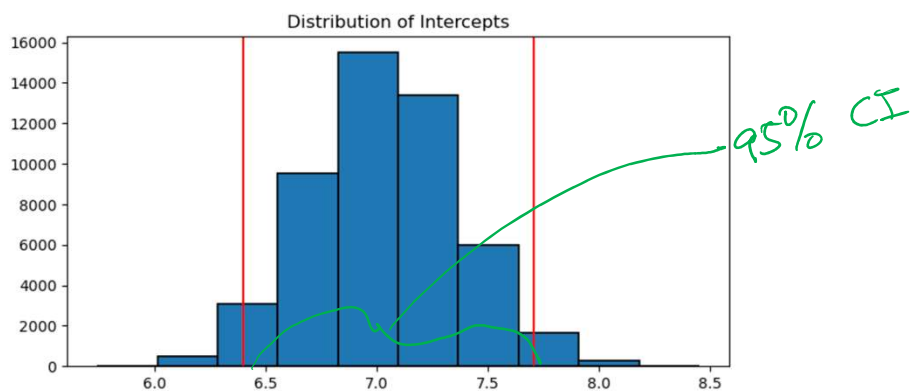
```
plt.figure(figsize=(8, 4))

plt.hist(bootstrap_intercepts, edgecolor='black', linewidth=1.2)
plt.title("Distribution of Intercepts")

conf_interval = np.percentile(bootstrap_intercepts, [2.5, 97.5])
print(conf_interval)

plt.axvline(conf_interval[0], color="red")
plt.axvline(conf_interval[1], color="red");

[6.398 7.707]
```



```
plt.figure(figsize=(8, 4))

plt.hist(bootstrap_slopes, edgecolor='black', linewidth=1.2)
plt.title("Distribution of Slopes")

conf_interval = np.percentile(bootstrap_slopes, [2.5, 97.5])
print(conf_interval)

plt.axvline(conf_interval[0], color="red")
plt.axvline(conf_interval[1], color="red");

[0.042 0.053]
```

